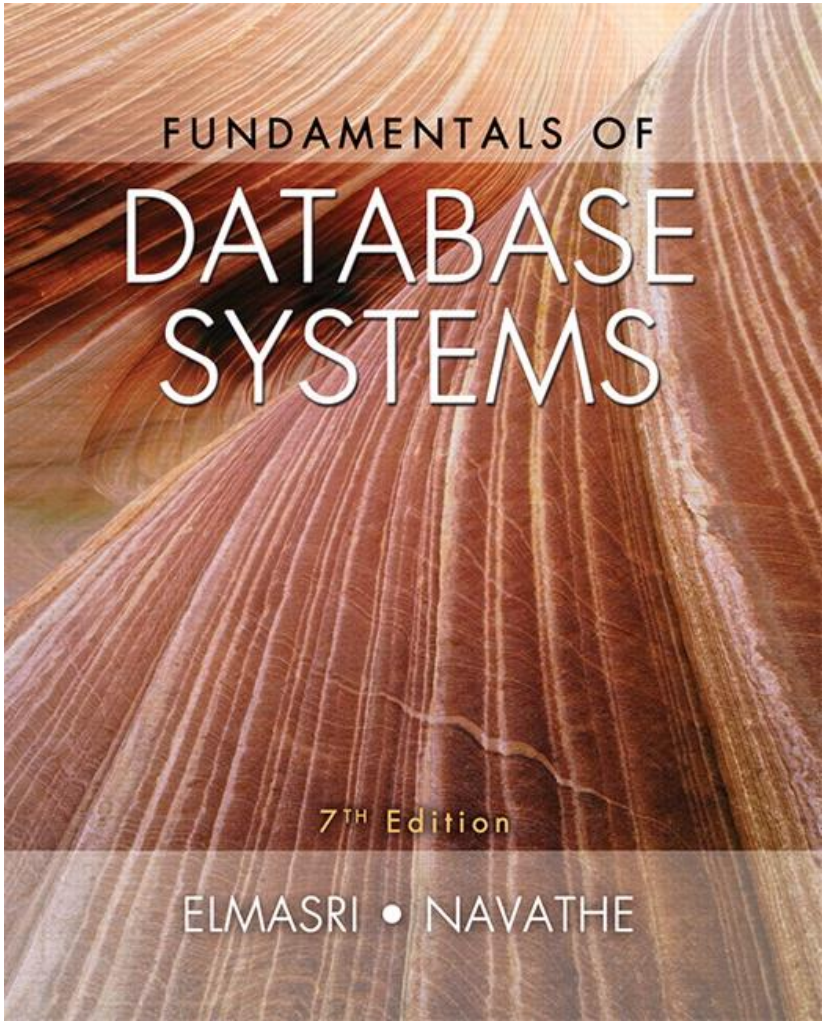
FUNDAMENTALS OF
DATABASE SYSTEMS
7TH Edition
ELMASRI • NAVATHE

Physical Database Design: CHAPTER 16

Disk Storage, Basic File Structures, Hashing, and Modern Storage Architectures

Introduction

- Databases typically stored on magnetic disks
  - Accessed using physical database file structures
- Storage hierarchy
  - Primary storage
    - CPU main memory, cache memory
  - Secondary storage
    - Magnetic disks, flash memory, solid-state drives
  - Tertiary storage
    - Removable media

Memory Hierarchies and Storage Devices

- Cache memory
  - Static RAM
  - DRAM
- Mass storage
  - Magnetic disks
    - CD-ROM, DVD, tape drives
- Flash memory
  - Nonvolatile

Storage Types and Characteristics

Type	Capacity*	Access Time	Max Bandwidth	Commodity Prices (2014)**
Main Memory- RAM	4GB–1TB	30ns	35GB/sec	$100–$20K
Flash Memory- SSD	64 GB–1TB	50μs	750MB/sec	$50–$600
Flash Memory- USB stick	4GB–512GB	100μs	50MB/sec	$2–$200
Magnetic Disk	400 GB–8TB	10ms	200MB/sec	$70–$500
Optical Storage	50GB–100GB	180ms	72MB/sec	$100
Magnetic Tape	2.5TB–8.5TB	10s–80s	40–250MB/sec	$2.5K–$30K
Tape jukebox	25TB–2,100,000TB	10s–80s	250MB/sec–1.2PB/sec	$3K–$1M+

*Capacities are based on commercially available popular units in 2014.

**Costs are based on commodity online marketplaces.

Table 16.1 Types of Storage with Capacity, Access Time, Max Bandwidth (Transfer Speed), and Commodity Cost

Storage Organization of Databases

- Persistent data
  - Most databases
- Transient data
  - Exists only during program execution
- File organization
  - Determines how records are physically placed on the disk
  - Determines how records are accessed

Secondary Storage Devices

- Hard disk drive
- Bits (ones and zeros)
  - Grouped into bytes or characters
- Disk capacity measures storage size
- Disks may be single or double-sided
- Concentric circles called tracks
  - Tracks divided into blocks or sectors
- Disk packs
  - Cylinder

Single-Sided Disk and Disk Pack

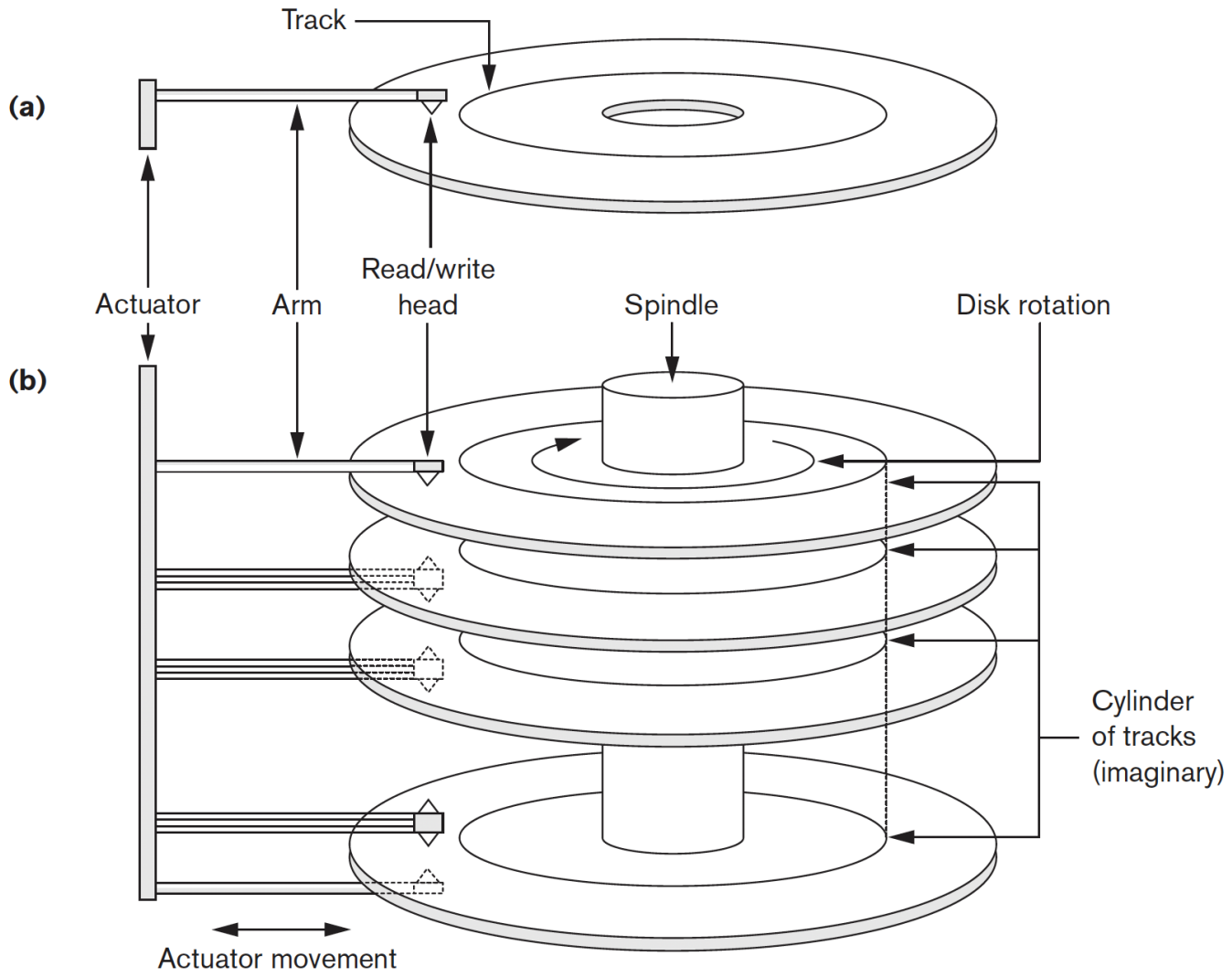

Figure 16.1
(a) A single-sided disk with read/write hardware. (b) A disk pack with read/write hardware.

Sectors on a Disk

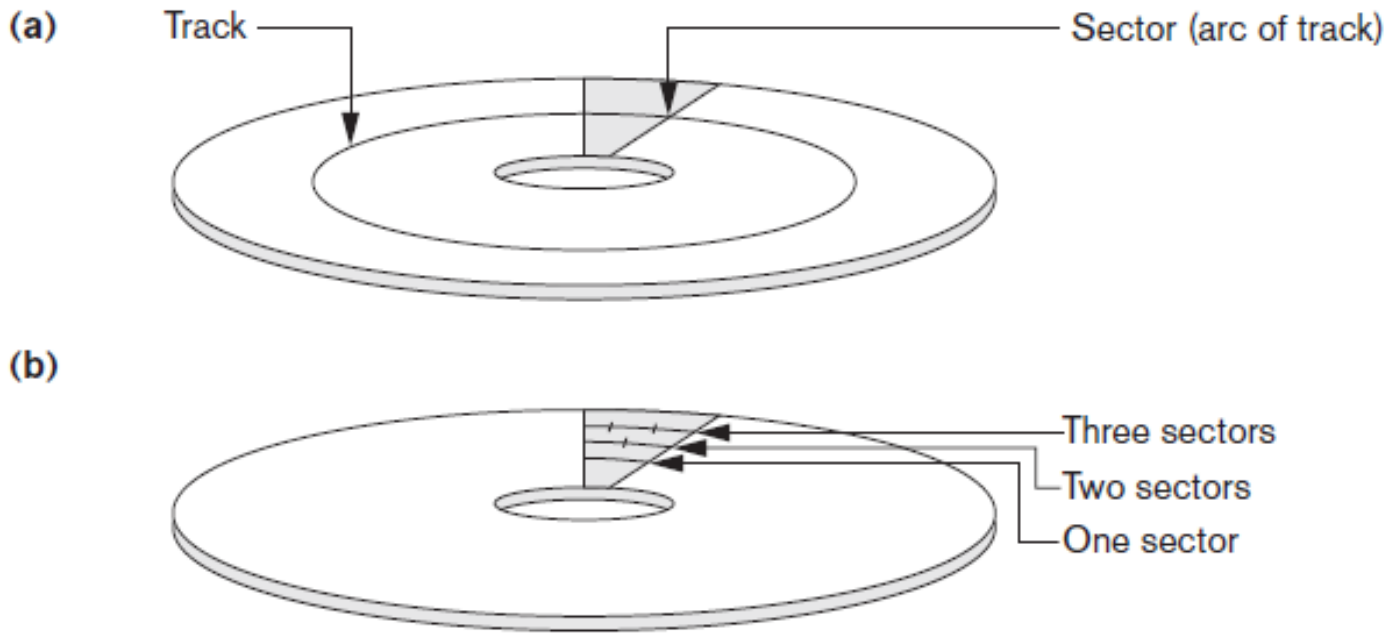

Figure 16.2 Different sector organizations on disk (a) Sectors subtending a fixed angle (b) Sectors maintaining a uniform recording density

Secondary Storage Devices (cont'd.)

- Formatting
  - Divides tracks into equal-sized disk blocks
  - Blocks separated by interblock gaps
- Data transfer in units of disk blocks
  - Hardware address supplied to disk I/O hardware
- Buffer
  - Used in read and write operations
- Read/write head
  - Hardware mechanism for read and write operations

Secondary Storage Devices (cont'd.)

- Disk controller
  - Interfaces disk drive to computer system
  - Standard interfaces
    - SCSI
    - SATA
    - SAS

Secondary Storage Devices (cont'd.)

- Techniques for efficient data access
  - Data buffering
  - Proper organization of data on disk
  - Reading data ahead of request
  - Proper scheduling of I/O requests
  - Use of log disks to temporarily hold writes
  - Use of SSDs or flash memory for recovery purposes

16.3 Buffering of Blocks

- Buffering most useful when processes can run concurrently in parallel

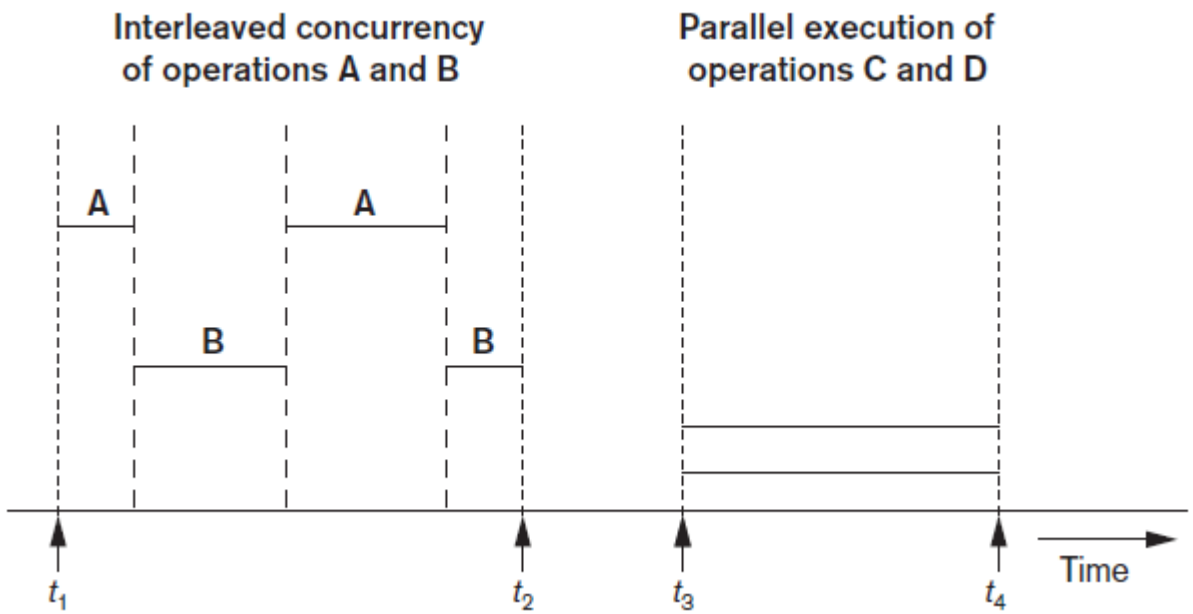

Figure 16.3 Interleaved concurrency versus parallel execution

Buffering of Blocks (cont'd.)

- Double buffering can be used to read continuous stream of blocks

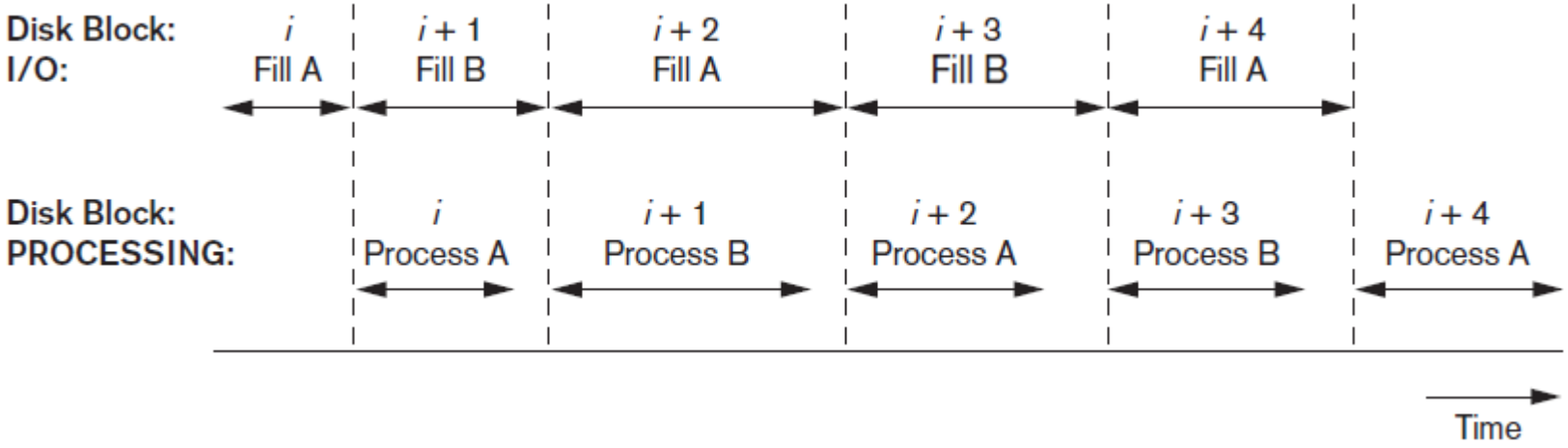

Figure 16.4 Use of two buffers, A and B, for reading from disk

Buffer Management and Replacement Strategies

- The term buffer is used to refer to a part of main memory that is available to receive blocks or pages of data from disk

- Buffer manager is a software component of a DBMS that responds to requests for data and decides what buffer to use and what pages to replace in the buffer to accommodate the newly requested blocks.

(1) to maximize the probability that the requested page is found in main memory, and

(2) in case of reading a new disk block from disk, to find a page to replace that will cause the least harm in the sense that it will not be required shortly again.

Buffer Management and Replacement Strategies

- The buffer manager views the available main memory storage as a buffer pool, which has a collection of pages.

- Buffer management information
  - Pin count: the number of times that page has been requested. If this count falls to zero, the page is considered unpinned.
  - Dirty bit: initially set to zero for all pages but is set to 1 whenever that page is updated by any application program.

Buffer Management and Replacement Strategies

When a certain page is requested, the buffer manager checks if the requested page is already in a buffer in the buffer pool; if so, it increments its pin-count. If the page is not in the buffer pool, the buffer manager does the following:

a. It chooses a page for replacement, using the replacement policy, and increments its pin-count.

b. If the dirty bit of the replacement page is on, the buffer manager writes that page to disk by replacing its old copy on disk. If the dirty bit is not on, this page is not modified and the buffer manager is not required to write it back to disk.

c. It reads the requested page into the space just freed up.

d. The main memory address of the new page is passed to the requesting application.

Buffer Management and Replacement Strategies

- Buffer replacement strategies
  - Least recently used (LRU)
  - Clock policy
  - First-in-first-out (FIFO)

16.4 Placing File Records on Disk

- Record: collection of related data values or items
  - Values correspond to record field
- Data types
  - Numeric
  - String
  - Boolean
  - Date/time
- Binary large objects (BLOBs)
  - Unstructured objects

Placing File Records on Disk (cont'd.)

- Reasons for variable-length records
  - One or more fields have variable length
  - One or more fields are repeating
  - One or more fields are optional
  - File contains records of different types

Record Blocking and Spanned Versus Unspanned Records

File records are allocated to disk blocks

- Spanned records
  - Larger than a single block
  - Pointer at end of first block points to block containing remainder of record
- Unspanned
  - Records not allowed to cross block boundaries

Record Blocking and Spanned Versus Unspanned Records (cont'd.)

- Blocking factor
  - Average number of records per block for the file

$$bfr = \lfloor B/R \rfloor$$

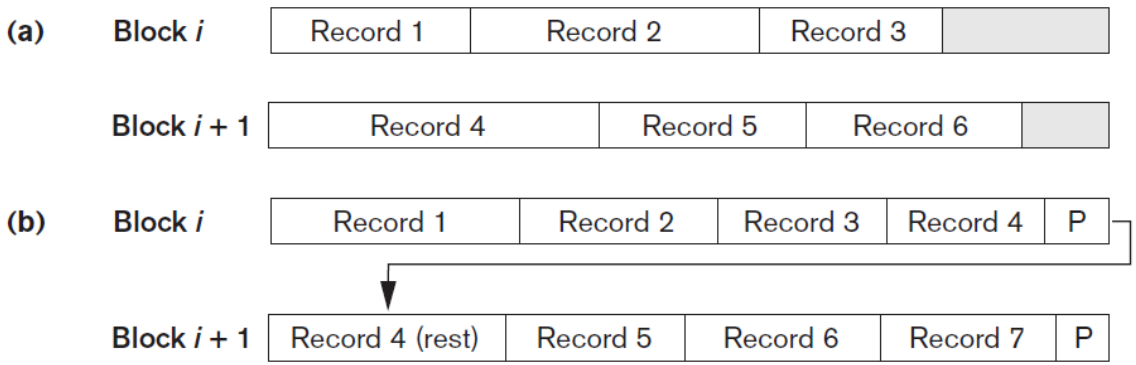

Figure 16.6
Types of record organization.
(a) Unspanned.
(b) Spanned.

Figure 16.6 Types of record organization (a) Unspanned (b) Spanned

Record Blocking and Spanned Versus Unspanned Records (cont'd.)

- Allocating file blocks on disk
  - Contiguous allocation
  - Linked allocation
  - Indexed allocation
- File header (file descriptor)
  - Contains file information needed by system programs
    - Disk addresses
    - Format descriptions
  - To search for a record on disk, one or more blocks are copied into main memory buffers. Programs then search for the desired record or records within the buffers, using the information in the file header.

16.5 Operations on Files

- Retrieval operations
  - No change to file data
- Update operations
  - File change by insertion, deletion, or modification
- Records selected based on selection condition

Operations on Files (cont'd.)

- Examples of operations for accessing file records
  - Open
  - Find
  - Read
  - FindNext
  - Delete
  - Insert
  - Close
  - Scan

16.6 Files of Unordered Records (Heap Files)

- Heap (or pile) file
  - Records placed in file in order of insertion
- Inserting a new record is very efficient
- Searching for a record requires linear search
- Deletion techniques
  - Rewrite the block
  - Use deletion marker
  - require periodic **reorganization** of the file to reclaim the unused space

16.7 Files of Ordered Records (Sorted Files)

- Ordered (sequential) file
  - Records sorted by ordering field
    - Called ordering key if ordering field is a key field
- Advantages
  - Reading records in order of ordering key value is extremely efficient
  - Finding next record
  - Binary search technique

Algorithm 16.1. Binary Search on an Ordering Key of a Disk File

```
l ← 1; u ← b; (*b is the number of file blocks*)
while (u ≥ l ) do
    begin i ← (l + u) div 2;
    read block i of the file into the buffer;
    if K < (ordering key field value of the first record in block i )
        then u ← i − 1
    else if K > (ordering key field value of the last record in block i )
        then l ← i + 1
    else if the record with ordering key field value = K is in the buffer
        then goto found
    else goto notfound;
    end;
goto notfound;
```

Access Times for Various File Organizations

Type of Organization	Access/Search Method	Average Blocks to Access a Specific Record
Heap (unordered)	Sequential scan (linear search)	$b/2$
Ordered	Sequential scan	$b/2$
Ordered	Binary search	$\log_2 b$

Table 16.3 Average access times for a file of *b* blocks under basic file organizations

16.8 Hashing Techniques

Another type of primary file organization is based on hashing, which provides very fast access to records under certain search conditions.

- This organization is usually called a **hash file**. The search condition must be an equality condition on a single field, called the **hash field**. In most cases, the hash field is also a key field of the file, in which case it is called the **hash key**.

16.8 Hashing Techniques

- The idea behind hashing is to provide a function *h*, called a **hash function** or **randomizing function**, which is applied to the hash field value of a record and yields the *address* of the disk block in which the record is stored.

- Hashing can also be used as an internal search structure
  - Used when group of records are accessed exclusively by one field value

Hashing Techniques (cont'd.)

- Internal hashing
  - Hash table: a data structure that stores key-value pairs
  - Hash function takes a key as input and produces an integer that is used as an index in the array.
- Collision
  - Hash field value for inserted record hashes to address already containing a different record
- Collision resolution
  - Open addressing
  - Chaining
  - Multiple hashing

Hashing Techniques (cont'd.)

- External hashing for disk files
  - Target address space made of buckets
  - Bucket: one disk block or contiguous blocks
- Hashing function maps a key into relative bucket
  - Table in file header converts bucket number to disk block address
- Collision problem less severe with buckets
- Static hashing
  - Fixed number of buckets allocated

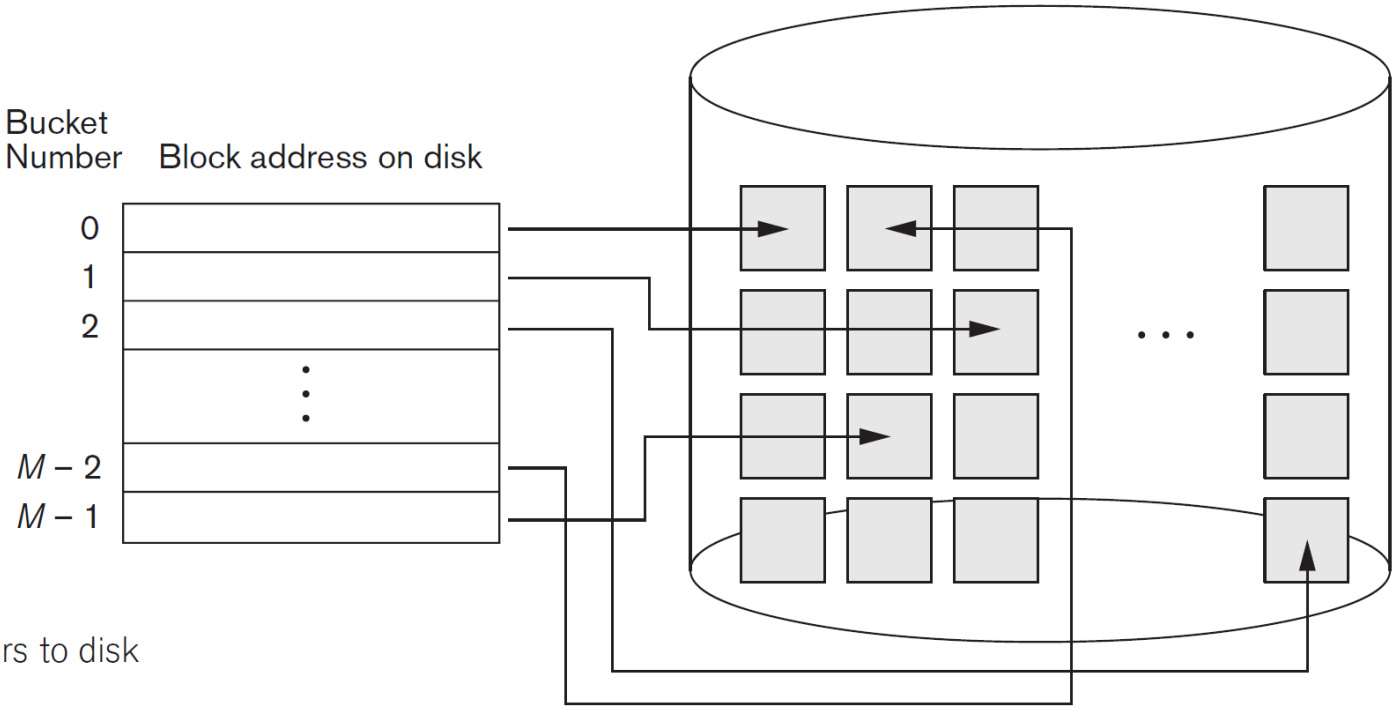

Figure 16.9
Matching bucket numbers to disk block addresses.

Hashing Techniques (cont'd.)

- Hashing techniques that allow dynamic file expansion
  - Extendible hashing
    - File performance does not degrade as file grows
  - Dynamic hashing
    - Maintains tree-structured directory
  - Linear hashing
    - Allows hash file to expand and shrink buckets without needing a directory

16.9 Other Primary File Organizations

- Files of mixed records
  - Relationships implemented by logical field references
  - Physical clustering
- B-tree data structure
- Column-based data storage

16.10 Parallelizing Disk Access Using RAID Technology

- Redundant arrays of independent disks (RAID)
  - Goal: improve disk speed and access time
- Set of RAID architectures (0 through 6)
- Data striping
  - Bit-level striping
  - Block-level striping
- Improving Performance with RAID
  - Data striping achieves higher transfer rates

Parallelizing Disk Access Using RAID Technology (cont'd.)

- Improving reliability with RAID
  - Redundancy techniques: mirroring and shadowing
- RAID organizations and levels
  - Level 0
    - Data striping, no redundant data
    - Spits data evenly across two or more disks
  - Level 1
    - Uses mirrored disks

Parallelizing Disk Access Using RAID Technology (cont'd.)

- RAID organizations and levels (cont'd.)
  - Level 2
    - Hamming codes for memory-style redundancy
    - Error detection and correction
  - Level 3
    - Single parity disk relying on disk controller
  - Levels 4 and 5
    - Block-level data striping
    - Data distribution across all disks (level 5)

Parallelizing Disk Access Using RAID Technology (cont'd.)

- RAID organizations and levels (cont'd.)
  - Level 6
    - Applies P+Q redundancy scheme
    - Protects against up to two disk failures by using just two redundant disks
- Rebuilding easiest for RAID level 1
  - Other levels require reconstruction by reading multiple disks
- RAID levels 3 and 5 preferred for large volume storage

RAID Levels

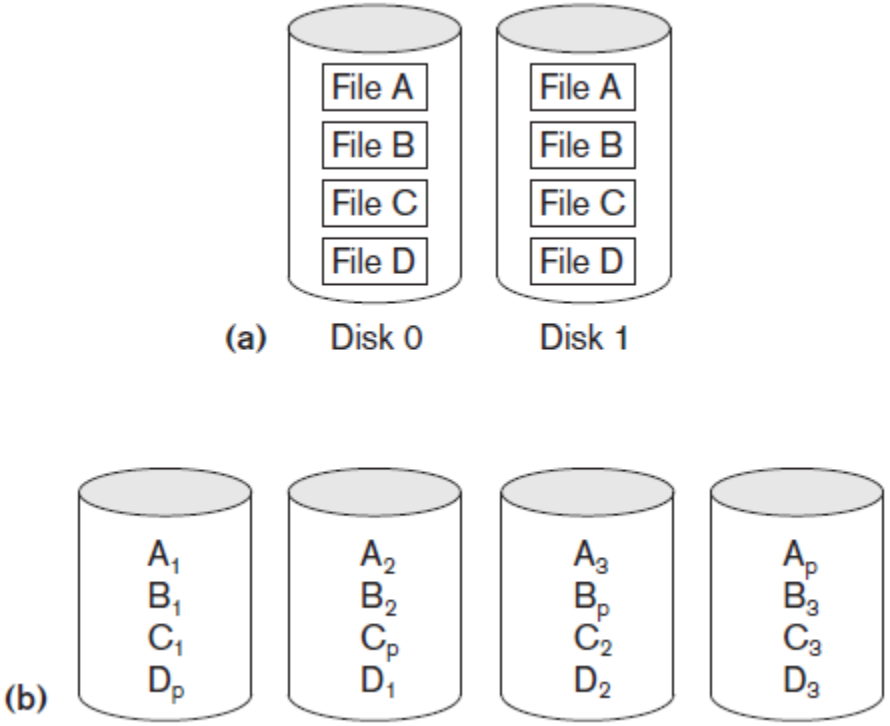

Figure 16.14 Some popular levels of RAID (a) RAID level 1: Mirroring of data on two disks (b) RAID level 5: Striping of data with distributed parity across four disks